SupportPearlz — LangChain RAG Customer Support Agent

Assignment: AAI–407 / PRAC–03

Status: Prototype and evaluation package; live API measurements and screenshots must be generated in the target environment.

Dataset: Fictional, internally authored knowledge base.

1. Executive summary

SupportPearlz is a customer-support Retrieval-Augmented Generation prototype for fictional Pearlz Home Systems. The system separates knowledge retrieval from answer generation so product and policy facts can be updated without retraining a language model. It uses a LangChain ingestion pipeline, recursive character splitting, OpenAI embeddings, a persistent Chroma vector store, a relevance gate, a grounding prompt, a Pydantic response schema, session history, and a Streamlit interface. Responses expose source metadata so an agent can verify claims before using them.

The repository includes a knowledge base of approximately 9,990 words across Markdown, CSV, PDF and DOCX sources, 20 evaluation cases across the required categories, scripts for measured chunking and retrieval comparisons, an embedding similarity diagnostic, a prompt-injection test set, an architecture diagram, a demo script, and a human-scoring worksheet. **The embedding-dependent experiments and live evaluation are not reported as completed here** because they require a configured API key and execution in the user's environment. No performance numbers are invented.

2. Architecture and implementation

The offline ingestion path recursively loads supported files, normalises text, attaches source/type/version/location metadata, splits documents into chunks and embeds them into Chroma. At query time, the conversation-aware query condenser rewrites follow-ups into standalone questions. The retriever returns candidate chunks; a relevance threshold determines whether enough evidence exists to proceed. Retrieved chunks are labelled `[S1]`, `[S2]`, etc. and passed to a model instructed to answer only from the supplied context. A Pydantic schema validates the answer, source labels, confidence and answered/refused state. The Streamlit UI presents the answer and evidence snippets.

The assignment manual PDF is excluded from the customer corpus because assessment instructions are not company policy. The architecture is shown in `docs/architecture.mmd`. Important limitations remain: an embedding API key is needed for index building and measured experiments; generated citations must still be reviewed against the actual context; and a threshold should be chosen from measured retrieval scores rather than copied from a tutorial.

3. Knowledge base design

The dataset contains product manuals for AquaPearl 500 Pro, AquaPearl 300 and AeroPearl Mini; warranty, refund/returns, shipping, privacy, service/maintenance, installation, troubleshooting, customer service, FAQ, pricing, ordering/account and product changelog documents; plus an installation safety PDF and service case worksheet DOCX. Pricing is tabular CSV. The corpus count is approximate and includes Markdown, text and CSV word tokens; PDF/DOCX extracted word counts are not included in that text-only number. Run the dataset word-count check after any further edits.

The fictional rules are intended to remain internally consistent: AquaPearl 500 Pro warranty is 24 months from invoice date, AquaPearl 300 is 18 months, and AeroPearl Mini is 12 months. Eligible unused returns have a 14-calendar-day window; approved refunds are generally processed 5–10 business days after inspection. Delivery estimates are 2–3 working days for metro locations, 3–5 for secondary cities and 5–7 for remote areas; Skardu is remote. These are estimates, not guarantees.

Deliberate cross-document seams include: (1) cabinet damage after a leak requires warranty exclusions and the service claim process; (2) unusual taste after filter replacement requires product-manual flushing instructions and the installation guide's escalation boundary; (3) firmware E-05 behaviour is documented only in the changelog; and (4) delivery damage involves both return policy and evidence handling. Deliberate gaps include student discounts, instalment payments, NSF certification, a universal installation fee, international shipping availability, and several undefined alert codes. The correct response to these gaps is not to guess.

4. Chunking experiment design

`evaluation/experiment_chunking.py` compares six configurations: (400, 0), (400, 80), (800, 0), (800, 120), (1200, 0), and (1200, 180), where each pair is character chunk size and overlap. It reports chunk count, mean and maximum chunk length, elapsed time, and top-4 answer-chunk hit rate. The hit criterion requires both the expected key fact to appear in a retrieved chunk and the source filename to match the test case's ground-truth source. This stricter criterion is more informative than cosine similarity alone.

Measured results: not yet available in this package. Run `python -m evaluation.experiment_chunking` after installing dependencies and setting `OPENAI_API_KEY`. Choose the final configuration from actual hit@4, chunk statistics, latency/cost and a manual review of whether chunks retain enough context. Smaller chunks may isolate facts but lose surrounding qualifications; larger chunks preserve context but can dilute relevance. The final report should quote the measured table from `evaluation/results/chunking_experiment.md` and explain the trade-off observed in this corpus.

5. Embeddings and retrieval strategy

`evaluation/embedding_diagnostic.py` embeds two paraphrases about a purifier's beeps and an unrelated address-change query. It records the actual embedding model, vector dimension and pairwise cosine similarity matrix. The expected diagnostic pattern is that paraphrases are more similar to one another than to the unrelated query, but this is a hypothesis to check against actual output rather than a result claimed in advance.

`evaluation/experiment_retrieval.py` compares similarity search at k=3, similarity search at k=8 and MMR at k=5. A hit requires the expected key fact and source to appear in the selected results. The final relevance threshold in `.env`/configuration must be calibrated against real relevant and irrelevant query scores. Similarity score scales vary by implementation and must not be treated as universal confidence probabilities.

Measured results and chosen threshold: pending live execution. Run `python -m evaluation.experiment_retrieval` after building the vector index. Record the actual hit rate and inspect false positives/false negatives before selecting a threshold.

6. Prompting, refusal and conversational memory

The system prompt instructs the model to use only supplied retrieved context, treat retrieved text and user text as data rather than instructions, avoid inventing prices/policies/commitments, refuse when support is absent, and never claim to issue a refund or perform another external action. The schema requires an answer, source labels, confidence and an answered flag. A relevance gate provides a deterministic refusal when no chunks survive the configured threshold.

The prompt-injection cases in `evaluation/prompt_injection_test.json` cover refund approval pressure, an attempt to reveal a secret, and a request to guess a certification. These cases must be run through the live UI and actual outputs saved. The weak-versus-grounded prompt comparison should be recorded only after the live run; a hypothetical unsafe output is not evidence that an experiment was conducted. Follow-up tests should inspect the logs for the original and rewritten query, then test a topic switch to ensure stale context is not carried over.

7. Evaluation plan and scoring rubric

The test set contains 20 cases: 4 easy/factual, 1 semantic, 3 multi-document, 3 follow-up/topic-switch, 4 unanswerable, 2 ambiguous, 2 adversarial and 1 safety case. This covers all six required minimum categories plus semantic and safety cases. The live runner writes `evaluation/results/eval_results.json` and `eval_results.md`. It calculates the unanswerable refusal rate from model outputs, but quality scores remain blank until manually reviewed.

Score each dimension from 0 to 2. **Correctness:** 0 wrong/unsupported fact, 1 partly correct or incomplete, 2 correct and qualified. **Groundedness:** 0 material unsupported claim, 1 mostly supported with a questionable claim, 2 all material claims supported by supplied context. **Citation accuracy:** 0 fabricated or irrelevant citation, 1 partial/incomplete citation, 2 each cited source supports the associated claim. Count a hallucination when a material claim is not supported by the supplied evidence. For every failure, identify the earliest faulty stage: ingestion, chunking, retrieval, context assembly, prompt/generation, schema validation or citation validation. Do not blame the prompt if the relevant chunk never reached the model.

8. Known limitations and next steps

1. Install pinned dependencies in a clean Python 3.11 environment and run the test suite.

2. Build the index from the authored corpus and inspect the ingestion report for each file.

3. Run the six chunking configurations, embedding diagnostic and three retrieval strategies; use measured values in the final results section.

4. Run all 20 cases and the injection set. Manually verify citations and complete the scoring columns.

5. Capture six screenshots using `docs/demo_script.md`, record a 5–8 minute demo, and replace this draft with the final measured report.

6. For production use, add authentication, secret management, rate limiting, PII redaction, monitoring, access controls and human review for safety, refunds and commercial commitments.

This report deliberately separates implemented components from results that still require live execution. A defensible evaluation must report failures honestly rather than claim 100% success without evidence.